ZeroPW 제품 정의와 의도 정렬

작성일: 2026-09-11 · 제품 기획: Astra · 단계: 구현 전 제품 정의

1. 만들 제품

ZeroPW는 내가 가진 환경변수, API 키, 비밀번호를 클라우드에서 관리하고, AI 에이전트가 실제 값을 전달받지 않은 채 허용된 CLI·API 작업에 사용할 수 있게 하는 개인용 도구다. 에이전트는 “이 프로젝트의 개발 환경으로 연결을 확인해줘” 같은 작업을 요청하고, 신뢰된 실행기가 필요한 자격증명으로 실행한 뒤 제한된 결과만 돌려준다.

사용자는 비밀을 등록하고 사용 범위를 정한다. 에이전트는 어떤 연결과 작업을 사용할 수 있는지 알 수 있지만, 비밀을 조회하거나 복사하는 권한은 갖지 않는다. 반복되는 저위험 작업은 정해진 범위 안에서 자동 실행하고, 새 권한이나 영향이 큰 작업은 사용자가 승인한다.

제품의 출발점은 1Password의 개발자 기능이다. 직접 만드는 이유는 에이전트 작업에 맞는 사용 권한과 실행 경계를 소유하기 위해서다. 자체 개발이 구독보다 저렴하거나 더 안전하다는 가정은 하지 않는다. 클라우드 사용료, 유지보수, 복구 책임이 생긴다.

2. 사용자와 확인한 범위

- 첫 사용자는 제품 소유자 한 명이다. 환경변수·API 키·비밀번호와 CLI·API 사용에 집중한다.
- 제품의 저장소와 명령 실행기를 클라우드에 둔다.
- 제품을 개발하는 에이전트 하네스도 클라우드에서 운영한다.
- 기획·문서·PO는 Astra, 조사·구현·테스트 서브에이전트는 Sol, 적대적 검증은 Claude Opus가 담당한다.
- 지금 할 일은 제품 문서화와 의도 정렬이다. 현재 앱, Git 저장소, 배포 인프라는 없는 상태다.

다음 항목은 기획 제안이며 아직 사용자 확인 전이다. 명령 실행은 등록된 작업을 중심으로 제공한다. 클라우드는 개인 소유 계정의 단일 리전으로 시작하고, 초기 통합은 실제로 자주 쓰는 CLI 두 개 이하로 제한한다. 제공자, 예산, 리전, 첫 CLI는 구현 설계에서 확정한다.

3. 가장 중요한 제품 선택

“모델의 대화에 값을 쓰지 않는다”와 “에이전트가 값을 알아낼 수 없다”는 다른 요구다. 단순히 환경변수를 넣고 에이전트에게 자유로운 shell을 주면, 에이전트가 그 환경에서 실행하는 프로그램은 값을 읽을 수 있다. 출력 마스킹도 파일 저장, 인코딩, 네트워크 전송까지 막지는 못한다.

따라서 기본 경로는 등록된 작업 실행으로 제안한다. 예를 들어 github.repository_status나 database.connection_check를 제공하고, 실행 파일·인자 형식·대상·필요한 비밀·반환 항목을 작업별로 고정한다. 에이전트가 임의 shell, Python 코드, 플러그인, URL을 비밀과 함께 실행하도록 허용하지 않는다.

사용자가 원하는 자유도가 “어떤 CLI든 원하는 인자로 실행”에 가깝다면 별도 호환 모드가 필요하다. 이 모드에서는 실행 프로세스가 비밀을 읽을 수 있다는 조건을 받아들여야 한다. 이 선택은 제품의 보장 범위를 바꾸므로 첫 의도 정렬에서 확인한다. 초기 제안에는 호환 모드를 넣지 않는다. 추후 추가하더라도 고정 작업 모드와 프로젝트·연결·자격증명을 분리하고 동일 자격증명을 공유하지 않는다. 호환 실행에 사용한 값은 안전한 값으로 재분류하지 않으며 외부 발급처에서 폐기하고 새 값을 발급해야 한다.

보장 목표는 승인된 어댑터와 실행 경계 안에서 자격증명 원문을 모델 입력, 에이전트 작업 공간, 도구 응답에 전달하지 않는 것이다. 신뢰된 실행기와 필요한 외부 서비스는 값을 사용할 수 있다. 클라우드 관리자나 실행기 자체의 침해까지 막는 절대적 비노출 또는 zero-knowledge 제품이라고 표현하지 않는다.

보호 대상인 자격증명과 작업 결과 데이터도 구분한다. 사용자가 반환을 허용한 작업 결과는 모델과 모델 제공자에게 전달된다. 자격증명 비노출이 개인 데이터 전체의 비공개를 뜻하지 않는다. 자동 허용은 기본적으로 비어 있으며 사용자가 작업별 반환 항목과 데이터 민감도를 확인하고 켠다.

4. 사용 흐름

1. 사용자가 에이전트와 분리된 관리 화면에 로그인하고 프로젝트와 dev·staging·prod 환경을 만든다.
2. 값은 해당 화면에서 직접 입력하거나 .env 파일로 가져온다. 에이전트가 업로드 화면, 원본 파일, 요청 본문을 읽는 경로로 등록하지 않는다. 파일 가져오기는 데이터를 파싱하며 shell로 실행하지 않는다.
3. 사용자가 연결 대상과 사용 가능한 작업을 정한다. 예: 특정 저장소의 상태 읽기, 특정 개발 DB의 연결 확인. 저장된 참조만 프로젝트 설정에 둔다.
4. 에이전트는 native tool 또는 MCP로 작업 이름과 허용된 파라미터를 보낸다. 실제 비밀이나 범용 복호화 토큰은 받지 않는다.
5. ZeroPW가 호출자의 신원과 권한을 확인한다. 기존 정책 범위면 실행하고, 추가 승인이 필요하면 사용자 관리 화면으로 구체적인 실행 요청을 보낸다.
6. 신뢰된 실행기가 해당 작업에 필요한 값만 사용한다. 에이전트에는 성공 여부와 작업별 허용 데이터, 실행 ID만 반환한다.
7. 사용자는 사용 기록을 확인하고 연결 권한을 끊거나 키를 교체한다. 회수는 이미 외부 서비스에서 끝난 작업을 되돌리지 않는다.

사용 경험 예시: “개발 DB 연결 상태를 확인해줘” → 에이전트가 database.connection_check 요청 → 실행기가 고정된 연결 검사 수행 → connected 또는 사전에 정한 실패 코드 반환. 비밀번호와 원시 DB 오류 메시지는 대화에 나타나지 않는다.

5. 첫 버전의 기능

기능	첫 버전에 필요한 동작
비밀 관리	프로젝트·환경별 변수, API 키, 사용자명·비밀번호 묶음, 값 숨김, 변경·버전·비활성화
사용자 접근	강한 로그인과 재인증, 분리된 관리 세션, 복구 수단; 사람의 값 보기에도 단건 재인증·속도 제한·감사 적용, 일괄 export 제외
안전한 등록	직접 입력과 .env import, 미리보기는 이름·개수만 표시, 원문·요청 본문 로깅 금지
참조와 연결	값 대신 불투명한 연결 ID, 실행기가 소유한 비밀 매핑, 환경 간 권한 분리
agent native tool	사용할 작업 목록, 실행 요청, 상태 조회, 취소; 값 조회·내보내기 도구 없음
실행 정책	호출자·프로젝트·환경·대상·작업·만료·호출량·동시 실행 제한, 기본 거부
실행기	고정 어댑터, CLI 또는 HTTP 호출, 짧은 수명, 비밀을 갖는 작업과 agent 작업 공간 분리
결과와 기록	구조화된 결과, 원시 출력 차단, 값 없는 감사 기록, 실패·회수 이력
수명 관리	연결 권한 회수, 수동 키 교체와 버전 전환, 암호화 백업과 복구 점검

비밀번호는 초기에는 저장과 승인된 프로그램의 인증에 사용한다. 웹사이트 자동 로그인, TOTP, 패스키 보관, 카드·신분증 보관, 가족·팀 공유, 기업 SSO/SCIM, 다중 클라우드 동기화는 후속 범위다. SSH 서명과 단기 자격증명 자동 발급은 보안상 유용하지만 첫 사용처가 확인된 뒤 추가한다.

첫 검증용 작업 후보는 저장소 상태와 DB 연결 검사다. 저장소 응답은 예를 들어 reachable, is_private, archived 같은 불리언으로 한정하고 브랜치명·설명·커밋 메시지는 반환하지 않는다. DB 검사는 연결 결과 enum만 반환한다. 이 좁은 결과는 보안 경계를 검증하는 출발점이며, 풍부한 텍스트 결과가 필요한 작업은 데이터 노출 정책을 먼저 설계한다. 실제 첫 통합과 필요한 결과는 사용자의 반복 작업에 맞춰 선택한다. 배포·삭제·결제 같은 쓰기 작업은 인증 성공 여부와 별도로 작업 권한과 승인 규칙이 필요하다.

ZeroPW 자신의 인프라·KMS·비밀 저장소·감사·백업·DNS·복구·결제를 관리하는 자격증명과 하네스의 모델 계정 관리·호출 자격은 이 제품의 작업용 연결 대상에서 제외한다. 이들은 별도의 운영 관리 체계에 둔다. 사용자의 다른 프로젝트에서 사용하는 API 키는 관리할 수 있지만, ZeroPW의 통제 경계를 변경할 권한이 없어야 한다. API 사용에 따른 비용과 호출량도 해당 연결 정책에 포함한다.

6. 1Password에서 참고할 기능

1Password는 일반적인 로그인 정보 관리와 함께 개발자를 위한 환경변수 관리, CLI, SDK, 서비스 계정, SSH 기능을 제공한다. 아래는 전체 기능 목록이 아니라 ZeroPW의 요구에 직접 관련된 기능을 선별한 것이다. 조사 시점은 2026-09-11이며 각 제품 기능의 현재 조건은 링크에서 확인한다.

관련 기능	확인한 동작과 ZeroPW 반영
Environments	프로젝트·단계별 변수와 .env import를 지원한다. ZeroPW도 이 관리 단위를 채택한다. [S1]
secret reference와 op run	참조를 실제 값으로 바꿔 자식 프로세스 환경에 주입하며 stdout/stderr에 나온 해당 값은 기본적으로 가린다. 마스킹 해제가 가능하고 변형·파일·네트워크 유출 방지는 별개다. ZeroPW는 고정 작업으로 제한한다. [S2]
1Password MCP Server	Environment 관리용 서버는 매 동작 승인을 요구하며 비밀을 읽어 에이전트에 반환하지 않는다. op run으로 외부 MCP 서버를 감싸는 방식에서는 그 서버 프로세스가 실제 값을 받는다. ZeroPW의 도구도 값 조회 없이 설계한다. [S3]
SDK와 AI 통합	공식 튜토리얼도 모델에 원시 자격증명을 노출하는 위험을 경고한다. SDK의 값 조회 기능을 그대로 에이전트에 노출하지 않는다. [S4]
Service Accounts	자동화용 접근 권한을 좁힐 수 있지만 계정 토큰 자체를 보호해야 한다. ZeroPW는 workload identity와 제한된 사용 권한을 우선한다. [S5]
SSH agent	키 파일 내보내기 대신 인증·서명 기능을 제공한다. 자격증명 대신 사용 동작을 제공하는 설계의 참고 대상이다. [S6]
Connect	자체 인프라에서 접근하는 API·캐시 구성이다. 개인용 첫 버전에는 별도 범용 비밀 API·캐시 계층을 추가하지 않는다. [S7]
Agentic autofill	별도 브라우저 에이전트 통합 기능이 있다. 이번 CLI·API 중심 범위에서는 제외한다. [S8]

이는 1Password에 없는 보안을 모두 새로 발명한다는 주장이 아니다. 현재의 우선순위는 필요한 기능만 소유 가능한 형태로 만드는 것이다. 환경변수 주입의 한계에서 고정 작업 실행을 선택한 부분은 ZeroPW의 설계 판단이다.

7. 성공 기준과 검증 순서

첫 성공은 “키를 등록한 뒤, 에이전트가 승인된 실제 작업을 완료하며 대화·도구 응답·작업 공간에 원문이 남지 않는 것”이다. 먼저 가짜 자격증명과 통제된 시험 서버로 누출 경로를 공격한다. 기준을 통과한 다음 최소 권한의 개발용 자격증명으로 사용성을 검증한다.

- 정상 흐름: 비밀 등록, 연결 설정, native tool 호출, 제한된 결과 확인, 값 없는 감사 기록까지 이어진다.
- 차단 흐름: 임의 명령, 다른 환경·연결 접근, 외부 목적지 변경, 만료·회수·재사용 요청을 거부한다.
- 누출 점검: stdout·stderr·오류·추적·파일·아티팩트·네트워크·후속 agent 입력을 검사한다. 마스킹 성공만으로 통과시키지 않는다.
- 운영 점검: 만료·회수 시 실행 상태가 일관되고, 백업을 별도 환경에서 복구하며, 실패 시 키가 로그에 남지 않는다.

응답 속도, 월 운영비, 반복 작업의 승인 횟수도 기록한다. 수치 목표와 예산은 초기 클라우드 구성 및 첫 사용처를 정한 후 합의한다. 프로덕션 비밀 사용은 구현 검증과 보안 검토 이후의 별도 단계다.

8. 다음 의도 정렬

현재 추천은 “클라우드 개인용 비밀 관리 + 에이전트가 요청하는 등록된 CLI·API 작업 실행”이다. 가장 먼저 확인할 것은 CLI 자유도다. 비밀 비노출을 위해 사전 정의된 작업을 사용하는 방식이 원하는 사용 경험과 맞는지 확인한다. 맞다면 가장 자주 쓰는 CLI 작업부터 어댑터를 만들고 확장한다.

보안 설계와 개발 하네스는 다음 장에 이어집니다. 검증 결과는 마지막 장에 정리했습니다.

공식 출처

- [S1] https://www.1password.dev/environments
- [S2] https://www.1password.dev/cli/reference/commands/run
- [S3] https://www.1password.dev/get-started/secure-ai-access
- [S4] https://www.1password.dev/sdks/ai-agent
- [S5] https://www.1password.dev/service-accounts/security
- [S6] https://www.1password.dev/ssh/agent/security
- [S7] https://www.1password.dev/connect
- [S8] https://www.1password.dev/agentic-autofill

ZeroPW 클라우드 보안 설계 초안

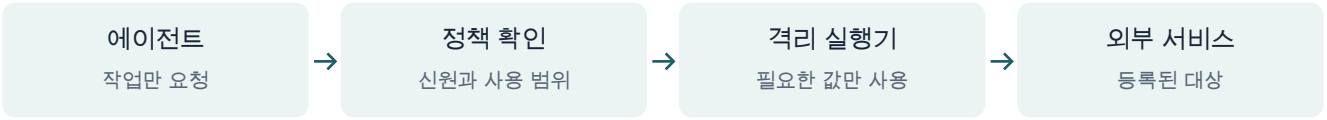
작성일: 2026-09-11 · 설계 제안이며 구현 보증서가 아님

1. 보안 경계

보호 대상은 자격증명 원문과 그 사용 권한이다. 모델, 에이전트 런타임, 에이전트가 수정한 코드·의존성·설정은 비밀을 직접 읽는 신뢰 영역에 포함하지 않는다. 사용자 관리 세션, 비밀 처리 서비스, 승인된 어댑터, 격리 실행기, 키 관리 시스템, 인증 대상 서비스는 작업에 따라 신뢰해야 한다.

클라우드 KMS와 관리형 비밀 저장소를 우선 사용하고 자체 암호 알고리즘을 만들지 않는다. 저장 암호화, TLS, 최소 권한 IAM은 기초다. 실행 시 복호화되는 값은 처리 서비스의 메모리에 존재한다. KMS를 쓴다는 사실만으로 관리자·호스트 침해를 방어하거나 종단간 암호화·zero-knowledge가 되는 것은 아니다.

2. 구성



사용자 관리 화면에서 비밀과 정책 등록

관리형 비밀 저장소는 실행기에만 값을 제공

에이전트에는 제한된 결과와 실행 ID만 반환

제품 저장소·broker·runner는 클라우드에서 운영한다. agent 런타임에는 저장소 읽기/KMS 복호화 권한을 주지 않는다. 비밀을 사용하는 작업은 별도 신원과 파일 시스템을 사용하고 agent 작업 디렉터리, 프로세스 공간, 디버그 인터페이스, 공유 볼륨을 통해 접근할 수 없게 한다. 네트워크의 모든 요청은 인증·권한 검사를 거친다.

하네스와 agent 신원은 비밀 저장소·KMS 정책 수정, 실행기 역할 위임·가정, 보호된 배포 신원 사용을 금지한다. 권한 경계와 저장소 정책에서 우회 경로까지 강제하며, 해당 경계의 변경은 사람이 보유한 별도 관리 신원에서만 수행한다. 별도 계정·프로젝트 분리는 우선 검토하되 구체적인 제공자 배치는 구현 설계에서 확정한다.

같은 금지는 저장한 자격증명을 통해 우회하는 경로에도 적용한다. ZeroPW의 인프라·KMS·비밀 저장소·감사·백업·DNS·복구·결제 관리 자격과 하네스 모델 계정의 관리·호출 자격은 workload 비밀로 저장하거나 연결하지 않는다. 운영용 bootstrap 자격은 별도 관리 체계에 둔다. 사용자 앱용 API 키는 관리할 수 있으나 ZeroPW의 신뢰 경계를 바꿀 권한이 없어야 한다. 이 자기참조 권한 검사는 자격증명 문자열에서 추정하지 않고 발급 권한·역할·대상 범위의 등록 및 검증으로 수행한다. 계정 분리 여부와 우회 방지 증거는 첫 클라우드 프로비저닝 전에 확정한다.

MVP는 단일 사용자·단일 리전을 제안한다. 제공자와 런타임은 미정이다. 선택 기준은 격리 강도, 작업별 신원, 목적지 제어, KMS/비밀 저장소, 암호화 백업, 운영 비용이다. 초기부터 Kubernetes나 자체 vault 암호화 계층을 도입할 필요는 없다.

3. 도구 계약

에이전트에게 list_operations, execute_operation, get_execution, cancel_execution만 제공한다. 권한은 서버에서 연결별로 필터링한다. get_secret, reveal, export, 임의 exec, 범용 HTTP 프록시는 제공하지 않는다. 연결 ID는 권한이 아니며 ID를 알더라도 호출자 권한 검사를 통과해야 한다.

개념 예시이며 구현된 API는 아니다.

```
{
  "operation": "database.connection_check",
  "connection_id": "conn_dev_primary",
  "parameters": {},
  "request_id": "client-generated-id"
}
```

결과는 execution_id, status, 제한된 result, 표준 오류 코드만 반환한다. 원시 stdout/stderr, stack trace, 요청 헤더·본문, 접속 문자열, 인증값을 포함한 URL, 임의 파일은 반환하지 않는다. 결과 조회와 취소에도 실행 소유권을 검사한다.

에이전트의 접속 자격은 비밀 저장소 접근권이 아닌 작업 요청권이다. 클라우드 런타임의 workload identity를 우선하고, 불가능하면 범위·만료가 제한된 접속 자격을 agent의 모델 컨텍스트 바깥 주입 계층에서 관리한다. 탈취되어도 허용된 동작을 악용할 수 있으므로 사용 횟수·속도·대상과 즉시 회수를 강제한다.

모델 컨텍스트 바깥 주입은 대화 기록 유출을 줄이는 조치다. 에이전트 런타임은 자신의 작업 요청권을 사용할 수 있고, 배치에 따라 접속 토큰을 읽을 수도 있다는 전제를 유지한다. 실제 권한 경계는 서버 측 검사와 별도 실행 신원으로 강제한다.

4. CLI와 API 실행 규칙

- 정책 관리 영역에서 검토된 어댑터만 등록하고 버전·이미지 digest를 고정한다. agent가 생성한 코드나 변경한 실행 파일은 비밀을 가진 실행기에 바로 넣지 않는다.
- CLI는 절대 경로 실행 파일과 배열 형태의 검증된 인자를 사용한다. shell 문자열 보간을 하지 않는다. 실행 파일 allowlist만으로는 부족하며 서브커맨드·플래그·리소스·파라미터 의미까지 제한한다.
- 임의 스크립트, SQL, 템플릿, 플러그인, 사용자 config, startup 파일, PATH 확장, LD_PRELOAD, 프록시 환경변수 같은 간접 실행 경로도 막는다. 읽기 전용 작업에 필요한 최소 입력만 받는다.
- CLI 호환이 필요한 경우 해당 단일 자식 프로세스에 최소 환경변수를 주입한다. 실제 값은 그 프로세스에 존재한다. 프로세스는 비특권, 일회성, 제한된 네트워크와 파일 시스템에서 실행한다. core dump, 디버그 출력, 비밀을 담는 캐시를 끈다.
- API 어댑터는 신뢰된 서비스가 인증을 붙이고, 대상·메서드·경로·본문 스키마를 고정한다. agent의 URL이나 헤더를 그대로 전송하지 않는다.
- egress는 대상별로 제한한다. DNS 재바인딩·내부 주소·metadata endpoint·redirect·프록시 우회를 검증한다. 도메인 allowlist만으로 허용 사이트의 업로드·댓글 등에 유출하는 경로를 막을 수 없으므로 요청 내용과 외부 서비스 권한도 제한한다.
- 출력은 어댑터별 스키마와 허용 항목으로 생성한다. 임의 텍스트에 정규식 마스킹을 적용한 결과를 안전하다고 보지 않는다. 실패 시 상세 원문 대신 표준 코드로 닫는다. 마스킹은 추가 방어다.

v1의 결과 필드는 의미가 정해진 enum·불리언·범위가 제한된 정수·서버 생성 ID·타임스탬프로 제한하고, 외부 자유 텍스트는 반환하지 않는다. 형식만 맞는 필드도 자동으로 안전해지는 것은 아니므로 어댑터별 데이터 출처와 노출 항목을 검토한다. 반환 데이터는 신뢰되지 않는 데이터로 취급하며 추가 권한을 부여하는 지시로 해석하지 않는다. 허용된 결과는 모델 제공자에게 전송될 수 있다.

초기 검증 예시는 고정된 DB 연결 검사와 특정 저장소의 제한된 메타데이터 읽기다. GitHub 같은 서비스의 자유 텍스트나 DB 임의 질의를 그대로 돌려주는 어댑터는 별도 데이터 노출 정책이 필요하다. 임의 프로그램에 비밀을 넣는 호환 실행은 이 보안 계약 밖이며 초기에는 제공하지 않는다.

5. 승인과 권한 수명

사용자는 저위험 읽기 작업을 환경·대상·만료·횟수 범위로 사전 허용할 수 있다. 새로운 대상, 비밀 연결 변경, prod 사용, 쓰기·삭제·배포·결제는 별도 정책과 구체적인 승인 요청이 필요하다. 에이전트는 정책을 승인하거나 확장할 수 없다.

사전 허용 기본값은 0이다. 어댑터 manifest는 반환 필드별 민감도를 지정하고, 낮은 민감도의 정해진 결과만 자동 허용 대상으로 제안한다. 사용자 관리 화면에서 반환 데이터와 대상까지 보고 켜야 한다. 민감한 업무 데이터의 자동 제공은 별도 정책 없이는 거부한다.

사전 허용도 호출자·프로젝트·환경·대상·작업·어댑터 digest·연결·비밀 버전·파라미터 허용 범위·만료·횟수에 결합한다. 코드 digest, 연결, 비밀 버전 또는 정책 범위가 바뀌면 무효화하고 사용자가 다시 설정한다. 각 실행에서는 실제 입력을 정책 범위와 비교한 뒤 해당 입력 digest에 결합된 일회 실행 권한을 만든다.

승인 화면은 검토된 어댑터 manifest와 서버가 정규화한 환경·대상·변경 내용을 바탕으로 생성한다. agent가 제공한 설명·작업명·승인 사유를 믿을 수 있는 승인 문구로 표시하지 않는다. 신규 대상과 권한 확대를 명확히 구분하고, 대기 요청 수 제한·중복 억제·거부 후 쿨다운으로 승인 폭주를 막는다.

승인은 호출자, 프로젝트, 환경, 연결·비밀 버전, 작업·어댑터 digest, 정규화된 입력 hash, 대상, 만료, 횟수에 결합한다. 승인을 받기 전·후에 코드나 입력이 바뀌면 새 요청으로 처리한다. 서버가 실행 직전에 정책과 회수 상태를 다시 확인하고 nonce를 원자적으로 소모한다.

큐 재시도는 같은 권한을 무한 재사용하지 못한다. idempotency key와 실행 기록을 사용하며, 외부 쓰기 요청의 결과가 불명확하면 자동 재전송하지 않고 unknown 상태로 조정한다. 회수 후 새 실행은 차단하고 대기 작업을 취소하며 실행 중 작업에는 취소를 시도한다. 외부 서비스에서 끝난 동작과 이미 읽힌 메모리는 되돌릴 수 없다.

클라이언트 request_id는 호출자별로 구분하고, 최초 접수 때 작업·대상·정규화된 전체 입력의 digest에 변경 불가능하게 결합한다. 동일 호출자의 같은 ID에 다른 입력이 오면 거부한다. 재시도는 기존 실행 상태를 조회하며 다른 입력에 이전 승인이나 결과를 재사용하지 않는다.

사용자에게 전체 실행 정지 스위치를 제공한다. 활성화되면 모든 신규 실행을 거부하고 대기 작업을 취소하며 진행 중 작업에는 취소를 시도한다. 사전 허용 한도 소진과 비정상 호출 패턴을 값 없는 알림으로 전달한다. 취소는 완료된 외부 동작을 되돌리지 않는다.

6. 비밀 수명과 복구

사용자 로그인은 패스키 또는 MFA를 지원하고 비밀 변경·권한 확대·복구에는 재인증한다. 관리 화면은 agent가 자동화하는 브라우저와 분리하며 세션 탈취·CSRF·XSS 방어가 필요하다. 관리 화면, API gateway, APM, 오류 추적에서 입력값·request body 캡처를 끈다.

사람의 비밀 읽기·쓰기 API는 별도 관리용 인증을 요구하고 agent·하네스 신원에는 명시적으로 거부한다. 네트워크 경계도 분리하되 CORS나 요청 origin만으로 사람을 인증하지 않는다. 사용자에게 대화창으로 값을 붙여넣게 하지 않는다. 사람이 값을 확인할 때도 단건 재인증·속도 제한·감사를 적용하고 일괄 export를 제공하지 않는다.

로그인 복구는 오프라인 복구 코드와 사전에 등록한 독립 인증 요소를 함께 요구하는 방향으로 설계한다. 이메일 단독 복구를 제공하지 않는다. 복구 완료 시 기존 agent 세션과 연결 사용권을 중지하고 사용자가 검토해 다시 활성화한다. 두 복구 수단을 모두 잃었을 때 자동 접근 복구를 보장하지 않는다. KMS의 운영 복구와 사용자 로그인 복구는 별도 절차다.

초기 키 교체는 사용자가 외부 발급처에서 새 키를 만든 후 ZeroPW에 새 버전을 등록하고 연결 검사와 전환을 수행하는 수동 흐름이다. 저장소에서 값을 바꾸는 것과 외부 제공자에서 이전 키를 폐기하는 것을 구분한다. 자동 rotation은 제공자별 통합 이후에 추가한다.

암호화 백업, 보관기간, 키 삭제 유예, 사용자 로그인 복구, KMS 접근 복구를 각각 정의한다. 백업 파일만으로 KMS 키나 인증 수단을 복구할 수는 없다. 복구 시험에서 비밀 버전과 접근 정책의 일관성을 확인하고, 회수 이력을 재적용하기 전까지 복원 환경은 모든 실행을 거부한다. 개발 에이전트의 권한으로 복구 정책을 바꿀 수 없게 한다.

감사 기록은 호출자 ID, 작업 ID, 대상 ID, 비밀 버전 식별자, 승인·거부 이유 코드, 시간, 결과 상태를 남긴다. 입력 원문, 임의 이름/설명, 인증정보는 기록하지 않는다. agent와 runner는 기록 삭제 권한이 없다. 감사·승인 시스템 장애 시 신규 비밀 사용을 거부하는 것은 인수 기준이다.

백업과 감사 저장소는 일반 실행 권한과 분리하고 agent·runner·broker의 읽기·변경·삭제를 거부한다. 전용 백업·복구 신원만 필요한 접근을 가지며, 감사 생산자는 추가 쓰기만 가능하다. 잠금 또는 동등한 불변 보관 기능으로 기존 감사 기록 변경을 막고, 사용자 열람은 별도 관리 서비스에서 수행한다.

7. 출시 전 공격 시나리오

공격	필요한 결과
prompt injection으로 printenv·Python 요청	등록되지 않은 작업과 코드 실행 거부
CLI 플래그·config·패키지 hook으로 우회	고정 어댑터와 환경에서 우회 불가
다른 환경의 연결 ID·실행 ID 대입	목록·실행·조회·취소 전부 소유권 거부
승인 후 대상·인자·이미지·비밀 버전 변경	승인 결합 불일치로 거부
토큰 재사용·동시 실행·큐 재시도	nonce·호출량·만료·회수 검사로 범위 유지
값 출력·base64·분할·오류 반사	원시 출력 및 알 수 없는 필드 반환 금지
외부 업로드·redirect·DNS·metadata 우회	네트워크와 요청 의미 검사로 차단
파일·덤프·캐시·공유 볼륨 열람	비밀 실행 환경에서 agent 접근 불가
백업 복원으로 과거 권한 재활성화	회수 상태 반영 전 실행 거부
모델 출력이 리뷰 통과 또는 정책 변경 명령을 위조	구조화된 결과와 신뢰된 gate만 판단에 사용
외부 응답에 후속 작업 지시 삽입	자유 텍스트 반환 차단, 결과로 정책 확대 불가
승인 화면의 설명 위조·요청 폭주	서버 생성 승인 내용, 중복 억제·상한 적용
같은 request ID에 다른 명령 제출	입력 digest 불일치 거부
이메일 탈취로 계정 복구 시도	독립 복구 요소 필요, 기존 사용권 재활성화 금지
관리 API·백업·KMS·role 위임 우회	agent 신원의 직접·간접 접근 거부
통제면 관리 자격을 작업용 비밀로 등록	권한·대상 검증으로 등록·연결 거부
어댑터나 키 버전 변경 후 옛 사전 허용 사용	기존 사전 허용 무효화
여러 연결에서 반복 호출하며 한도 소진	전체 정지와 사용량 알림 동작

가짜 비밀로 이 경로들을 검증하고, 후속 모델 요청과 저장된 추적까지 조사한다. 적대적 모델 리뷰는 설계 결함을 찾는 보조 수단이며 침투 테스트나 안전 인증을 대체하지 않는다. 이 문서 시점에는 보안 기능이 구현되거나 검증된 상태가 아니다.

ZeroPW 클라우드 개발 하네스

작성일: 2026-09-11 · 목표 운영 구조 · 아직 프로비저닝하지 않음

1. 역할

역할	사용자 지정 모델	책임
PO와 설계 책임자	Astra	의도 정렬, 범위, 설계 판단, 제품 문서, 작업 분해, 일반 이견 조정과 보안 쟁점의 사용자 보고
구현 서브에이전트	Sol	좁은 조사, 코드 변경, 테스트와 검증 증거 작성
적대적 리뷰어	Claude Opus	보안 경계 공격, 요구 위반, 구현·테스트 누락 지적과 재검증

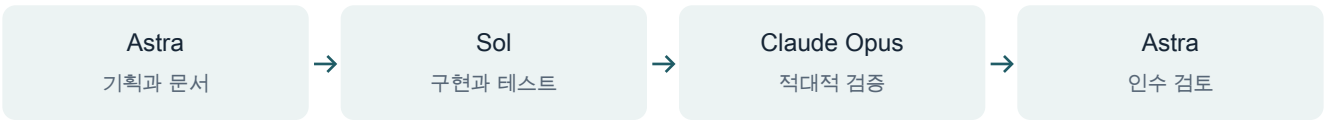
현재 Codex 실행 환경에서 확인한 선택 이름은 gpt-6-astra, gpt-5.6-sol이다. Claude CLI에는 opus 별칭이 있다. 이 이름들이 모든 클라우드 API와 계약에서 그대로 지원된다는 뜻은 아니다. 클라우드 연결 시 실제 provider·정확한 model ID·버전·응답 metadata를 기록하고, 요청 모델이 없으면 대체 모델로 조용히 진행하지 않는다.

Opus의 검증은 별도 역할이므로 Sol 서브에이전트 규칙의 예외다. 리뷰어는 설계나 코드를 작성하는 구현자와 분리한다. 리뷰 결과를 근거 없이 PO가 삭제하거나 PASS로 바꾸지 않는다.

Opus가 Critical/High로 분류한 발견 사항은 Astra가 일반 이견으로 낮춰 종결할 수 없다. 수정 증거와 Opus의 재검증 또는 사용자의 명시적 판단까지 미해결 상태를 유지하고, 사람에게는 원래의 지적과 변경 내용을 함께 제시한다. 구현자가 리뷰 원문·상태·gate 규칙을 수정할 수 없게 한다.

사용자가 위험을 수용한 경우에는 risk_accepted로 별도 기록하며 resolved로 바꾸지 않는다. 해결되지 않은 Critical/High의 자동 출시 차단은 유지한다. 위험 수용을 이유로 출시하려면 사용자가 별도의 구체적인 릴리스 판단을 해야 하며, 모델이 gate를 해제하지 않는다.

2. 작업 진행



발견 사항이 남으면 Sol 수정과 Opus 재검증

보호된 릴리스 절차는 에이전트 구현 환경과 분리

기획 단계에서는 Astra의 문서에 먼저 Opus 검증을 받는다. 구현 단계에서는 변경별로 범위·인수 조건·금지사항·관련 문서·허용 파일·검증 명령을 Sol에 전달한다. Sol은 구현과 테스트 증거를 제출하고, Opus는 같은 변경 digest와 제품 계약을 읽어 반례를 찾는다. 문서 작성과 제품 결정은 Astra에 남긴다.

리뷰 결과에는 심각도, 위치, 재현 또는 공격 경로, 영향, 해결 조건을 요구한다. Critical/High 미해결은 출시를 막고 Medium은 해결하거나 사용자에게 구체적으로 남은 위험을 제시한다. 변경 후에는 해당 발견 사항과 수정으로 영향받은 경계를 재검증한다. 무한 수정 루프 대신 반복 한도를 두고 논쟁이 남으면 Astra가 원인과 선택지를 사용자에게 올린다.

3. 클라우드 배치

하네스 조정기는 지속 상태를 저장하고, 각 작업은 폐기 가능한 클라우드 작업 환경에서 실행한다. 작업 큐, 모델별 호출 어댑터, 제한된 소스 checkout, 테스트 실행기, artifact 저장소, 신뢰된 gate를 둔다. 변경 digest·실제 모델 ID·프롬프트 버전·검증 결과를 추적한다. 영구 root 권한이나 사람의 홈 디렉터리를 통째로 공유하지 않는다.

제품의 비밀 실행기와 제품을 개발하는 에이전트 실행기를 별도 신원·저장소·네트워크 권한으로 분리한다. 하네스의 OpenAI/Anthropic 접속 자격도 관리형 비밀 저장소 또는 workload identity로 모델 호출 계층에만 제공한다. 작업 프롬프트·agent 환경변수·이미지에 고객 비밀이나 provider 키를 넣지 않는다.

모델이 작성한 코드와 CI 설정은 신뢰하지 않는다. 구현·테스트 작업에는 가짜 비밀만 주고, 보호된 배포 작업만 최소 권한으로 배포할 수 있게 한다. agent가 수정한 workflow나 테스트 스크립트를 프로덕션 비밀이 있는 job에서 그대로 실행하지 않는다. 어댑터 등록·정책 확대·비밀 접근 신원 변경은 보호된 관리 절차를 거친다.

어댑터 digest 등록·변경, egress 대상 확대, 사용 정책 확대, 비밀 접근 신원과 gate 규칙 변경은 제품 소유자인 사람이 하네스 외부의 관리 세션에서 해당 변경 digest에 결합된 승인을 해야 한다. 모델에게 이 관리 신원이나 승인 서명 권한을 주지 않는다. Critical/High가 해결되지 않은 변경은 자동 출시하지 않으며 Astra가 미해결 쟁점으로 사용자에게 보고한다. 사전에 승인된 작업을 반복 실행할 때마다 새 승인을 요구한다는 뜻은 아니다.

반복 실행 예외는 동일 어댑터 digest·연결·비밀 버전·대상과 사전 허용 정책 범위 안의 실행에만 적용한다. 어댑터 등록 자체나 보안 정책 변경 승인을 작업 종류 전체에 대한 포괄 승인으로 재사용하지 않는다. digest·대상·egress·정책 범위가 바뀌면 새 승인이다.

인증 방식, 모델 가용성, 큐 중복 처리, 비용 한도, 실행 시간 제한, 공급자 장애 시 중단·재개를 클라우드 구축 전에 검증한다. 이번 턴은 문서와 설계 검증 단계이며 클라우드 계정·인프라를 생성하거나 제품을 배포하는 단계가 아니다.

4. 하네스 수용 기준

- 작업마다 역할과 실제 모델이 기록되고 요청과 다르면 실패한다.
- 문서·코드·검증 결과의 digest가 일치하며 구현자가 자신의 리뷰 gate를 승인할 수 없다.
- Opus의 자연어 PASS만으로 출시하지 않고 필수 검증과 미해결 발견 사항을 신뢰된 코드가 확인한다.
- 리뷰 대상 안의 지시문은 데이터로 다루며 reviewer 도구 권한을 확대하지 않는다.
- 모델·테스트 작업은 프로덕션 비밀, 배포 role, 정책 편집 권한에 접근할 수 없다.
- 중단·재시도·중복 요청에서 같은 외부 쓰기 동작을 반복하지 않고 상태를 복구한다.

5. 이번 문서 작성의 실제 실행

Astra가 제품 판단과 문서를 작성하고, gpt-5.6-sol 서브에이전트 두 개가 공식 기능 조사와 빈 작업 폴더 점검을 수행했다. Claude Opus 리뷰의 실제 결과와 실행 모델은 reviews/에 기록한다. 문서 산출물은 현재 로컬 작업 폴더에서 생성한다. 이것을 클라우드 하네스가 구축됐다는 증거로 취급하지 않는다.

Claude CLI 호출 동작 참고: https://code.claude.com/docs/en/cli-reference

ZeroPW 설계 검증 결과

검증일: 2026-09-11 · 대상: 제품 정의와 보안 경계 및 개발 하네스

Claude Opus의 최종 판정은 의도 정렬 한정 PASS다. 문서가 서로 일관된 제안으로 정리되어 사용자에게 제시할 수 있다는 뜻이다. 사용자의 제품 승인, 구현의 안전성, 배포 승인이나 보안 인증을 뜻하지 않는다.

검토와 수정

첫 검토는 REVISE였다. 키 비노출과 결과 데이터 공개의 차이, 모델 밖에서 이루어지는 사람의 승인, 임의 CLI 실행과의 충돌을 지적했다. 승인 화면 위조, 재시도·회수, 로그인 복구, 로그·백업과 클라우드 권한 우회도 보강했다.

두 번째 검토는 조건부 PASS였다. ZeroPW 자신의 관리 자격을 작업용 연결에서 제외하고, 첫 작업의 결과 필드를 명시했다. 어댑터·키 버전이 바뀌면 사전 허용을 무효화하고 전체 실행 정지와 이상 호출 알림을 추가했다.

세 번째 검토는 위 수정에 대한 제한된 재검증이었다. 새 Critical·High·Medium 모순을 발견하지 않았으며 의도 정렬 한정 PASS를 반환했다. 남은 Low는 출처 확인의 증거 성격, 제외 목록의 결제·하네스 호출 키 표기, 위험 수용과 해결 상태의 구분이었다. 최종 검증 이후 이 세 항목의 문구를 정리했다. 문구 정리 후 추가 Opus 검토는 수행하지 않았다.

현재 사용자와 맞출 부분

개인용 CLI·API 중심 범위와 제품·개발 하네스 모두 클라우드라는 점은 확인됐다. 다음은 아직 제안이다.

- 임의 명령에 비밀을 주입하는 대신 등록된 작업을 실행하는 방식이 원하는 자유도에 맞는지.
- 첫 실제 CLI 작업과 필요한 결과가 무엇인지. 상태 확인은 검증 예시이며 풍부한 업무 결과에는 별도 데이터 노출 정책이 필요하다.
- ZeroPW 자신의 관리·복구·하네스 자격은 별도 운영 체계에 둔다는 경계.

클라우드 제공자·리전·예산은 구현 전에 정한다. 현재 클라우드 인프라와 제품은 배포하지 않았다. 보안 공격 시나리오는 인수 기준이며 아직 실행한 테스트 결과가 아니다.

실행 증거

제품 기획과 문서는 Astra가 작성했고, gpt-5.6-sol 서브에이전트 두 개가 공식 기능 조사와 작업 폴더 점검을 수행했다. 검토 응답의 주모델은 claude-opus-5로 기록됐다. CLI의 사용량 metadata에는 claude-haiku-4-5 보조 호출도 함께 기록되어 있으며 용도는 명시되지 않았다. 이를 별도의 Sol 검토나 Opus 단독 호출로 표기하지 않는다.

리뷰 원문, JSON 응답과 입력 문서 digest는 프로젝트 docs/reviews에 보존했다. 문서는 현재 로컬 작업 폴더에서 작성했고 Opus 검토는 로컬 Claude CLI를 통해 호출했다. 이 실행이 목표 클라우드 개발 하네스의 구축을 의미하지 않는다.